Homework 3

10 分满分

2026/4/20

第 1 次尝试

进行中

下一项：提交作业

添加评论

允许尝试 3 次

详细信息、细节

Week 4: Build a Math Agent with Tool Use

Overview

This assignment builds on the structured output and tool use concepts from Week 4. You will run a ReAct agent that solves problems using tools, observe how it fails when a tool is missing, implement the missing tool, and show that the agent now solves all problems correctly.

A starter repo is provided with a working agent and one tool (a calculator). The agent can solve pure math questions, but it will fail on questions that require looking up product prices from a catalog. Your job is to implement the product lookup tool so the agent can answer all 8 questions.

The starter uses **PydanticAI** (https://ai.pydantic.dev/), which handles the ReAct loop and works with any LLM provider. The starter defaults to **Google AI Studio** (https://aistudio.google.com) (free API key). You may use any other provider (OpenAI, Anthropic, etc.) by changing one line.

You may use a coding assistant such as **Claude Code**, **OpenCode**, **Codex**, or another instructor-approved alternative to help you write your code.

Starter repo

The starter repo is at: **https://github.com/harangju/bu-330-760-week4** (https://github.com/harangju/bu-330-760-week4)

It includes:

- `agent.py` - a ReAct agent with one working tool (calculator) and one tool to implement (product lookup)
- `calculator.py` - calculator tool
- `products.json` - product catalog with prices
- `math_questions.md` - 8 questions the agent will solve
- `.env.example` - template for your API key
- `pyproject.toml` - dependencies

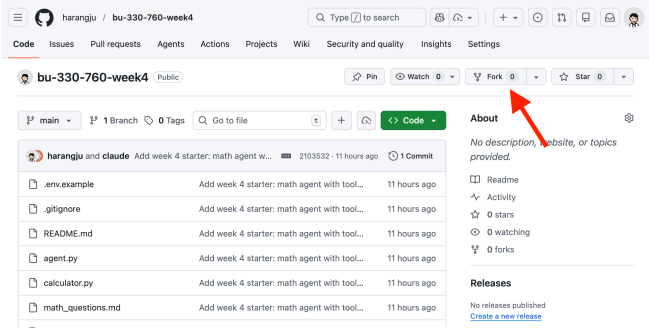
Review the files so you have context about what they do. In particular, read through `agent.py` and `calculator.py` closely: they show you how to define an agent with a system prompt and how to register a tool with `@agent.tool_plain`. You will follow the same pattern when you implement `product_lookup`.

Workflow

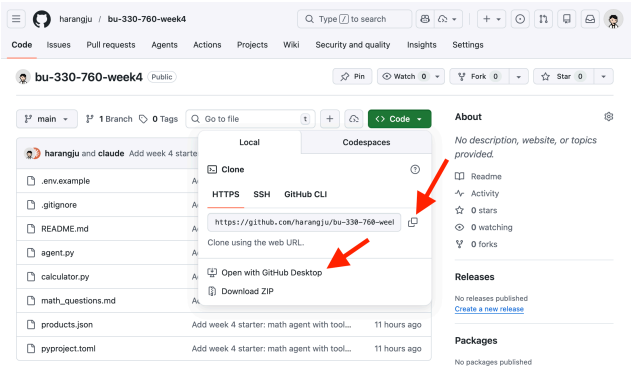
Work through the steps below in order.

Step 1) Fork, clone, and set up

Go to the **starter repo** (https://github.com/harangju/bu-330-760-week4) on GitHub and click the **Fork** button in the top right:



This creates a copy of the repo under your GitHub account. Then click the green **Code** button on your fork and copy the HTTPS URL:



Clone your fork locally using the command line:

```
git clone https://github.com/YOUR-USERNAME/bu-330-760-week4.git
cd bu-330-760-week4
```

Or use **GitHub Desktop** (https://desktop.github.com/) : click **File > Clone Repository**, pick your fork from the list, and choose where to save it.

Copy or rename `.env.example` to `.env` and add your API key. Get a free key at **Google AI Studio** (https://aistudio.google.com/apikey) , or use another provider (`OPENAI_API_KEY` , `ANTHROPIC_API_KEY` , etc.) by changing the `MODEL` line in `agent.py` and setting the matching key in `.env` :

```
cp .env.example .env
```

Then edit `.env` and replace `your-key-here` with your key. Make sure `.env` is in your `.gitignore` so you don't commit your key.

Step 2) Run the agent and observe what fails

Run the agent directly in your terminal (not inside a coding assistant):

```
uv run agent.py
```

Watch the output. Questions 1-4 (pure math) should work. Questions 5-8 (product prices) should fail because the agent has no way to look up prices from the catalog.

Step 3) Implement the product lookup tool

Open `agent.py` and find the `TODO` section. Implement the `product_lookup` tool so it reads `products.json` and returns the price for a given product name. If the product isn't found, it should return the list of available products.

The pattern is the same as `calculator_tool` above it.

Commit and push your work after this step.

Step 4) Run the agent again

Run the agent again in your terminal:

```
uv run agent.py
```

All 8 questions should now produce correct answers. The agent should use the calculator for math and the product lookup for prices.

Step 5) Record a walkthrough video

Create a **30-60 second screen recording**.

The easiest option is **Zoom cloud recording**: start a meeting by yourself, hit Record > Record to the Cloud, and share the resulting cloud link. If you care about privacy, Zoom has an avatar feature you may use.

Alternatively, upload the recording as an **unlisted YouTube video** (or **unlisted Vimeo** if YouTube is not practical).

Don't worry about getting the video perfect. It shouldn't take you more than 1-3 tries to record the video.

Show the agent running in your terminal (`uv run agent.py`), and narrate through what the agent is doing and the outputs that it generates. The viewer should be able to see the ReAct traces printing as the agent solves each question.

Add the video link to `README.md`. Then commit and push.

Final deliverables

Submit your GitHub repository link in Canvas. Your `README.md` must include the video link (Zoom cloud recording, unlisted YouTube, or unlisted Vimeo).

Your repository should contain:

```
hw4-yourname/
├─ README.md              (must include the video link)
├─ agent.py               (with product_lookup implemented)
├─ calculator.py
├─ products.json
├─ math_questions.md
├─ pyproject.toml
├─ .env.example
└─ .gitignore
```

What success looks like

A strong submission:

- has a working `product_lookup` tool that reads from `products.json`
- shows the agent solving all 8 questions correctly with visible ReAct traces
- includes a video of the agent running in the terminal

Submission

On Canvas, submit your GitHub repository link. The video link must be in your `README.md`.

输入 Web URL

https://

(https://jhu.instructure.com/courses/113744/modules/items/5572211)

(https://jhu.instructure.com/courses/113744/modules/ite